

Guide

Source data

In this example, I use H2 as the source database.

Use H2 to create relational database

Login link: <http://localhost:8082> or

[H2 Console](#)

Create dataset “allpatientinfo” as the demo dataset and “patientinfo” as the one track incremental data. The two tables have exactly the same structure.

The screenshot shows the H2 Console interface. The top bar includes a toolbar with icons for undo, redo, auto commit, and other functions. Below the toolbar, the left sidebar lists the database schema: ALLPATIENTINFO, DOCTORINFO0901, PATIENTINFO, PATIENTINFO0901, TEST, INFORMATION_SCHEMA, Users, and H2 2.3.232 (2024-08-11). The main area displays the command history and sample SQL script.

Important Commands

Icon	Description
?	Displays this Help Page
📜	Shows the Command History
🚀	Executes the current SQL statement
🔍	Executes the SQL statement defined by the text selection
⌘+Space	Auto complete
🔌	Disconnects from the database

Sample SQL Script

Action	SQL Statement
Delete the table if it exists	DROP TABLE IF EXISTS TEST;
Create a new table with ID and NAME columns	CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255));
Add a new row	INSERT INTO TEST VALUES(1, 'Hello');
Add another row	INSERT INTO TEST VALUES(2, 'World');
Query the table	SELECT * FROM TEST ORDER BY ID;
Change data in a row	UPDATE TEST SET NAME='Hi' WHERE ID=1;
Remove a row	DELETE FROM TEST WHERE ID=2;
Help	HELP ...

Adding Database Drivers

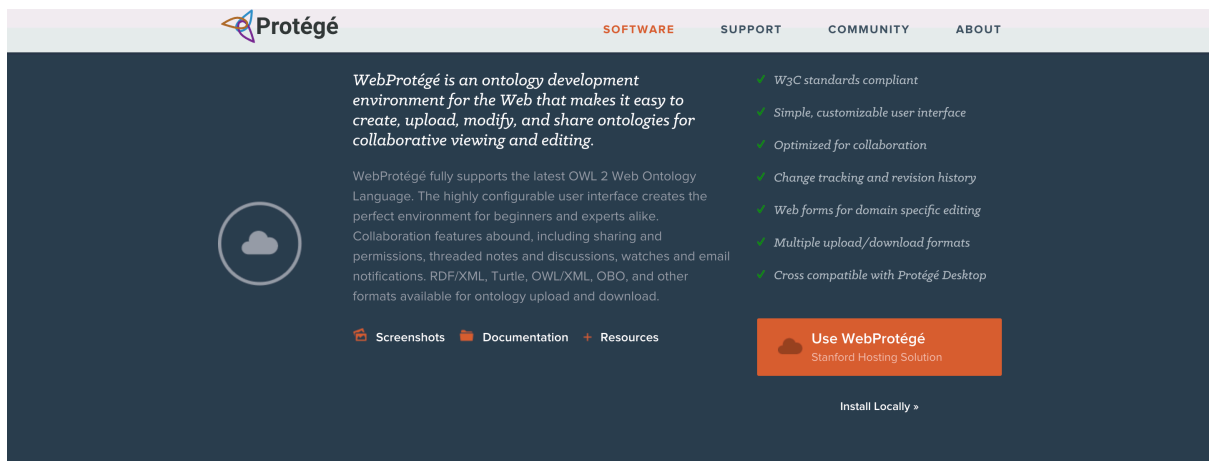
Activate Windows
Go to Settings to activate Windows.

Ontology

To validate the proposed ontology product, I use Protégé to create ontology. If users do not need new ontology, this step can be skipped.

Protégé link: <https://protege.stanford.edu/software.php#web-protege>

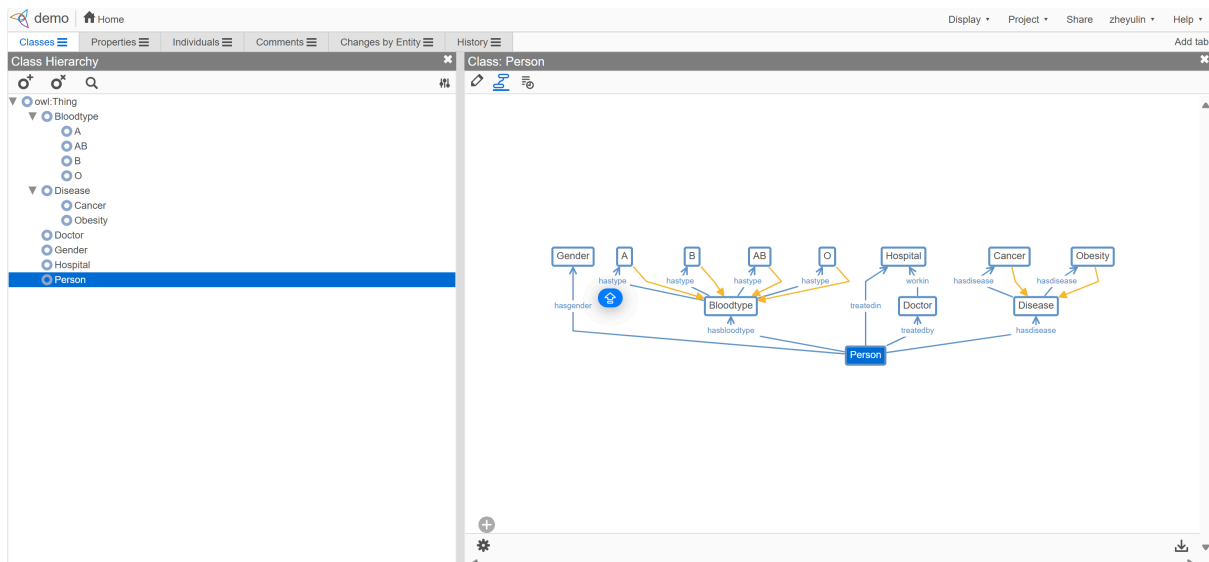
For convince, I use the web version.

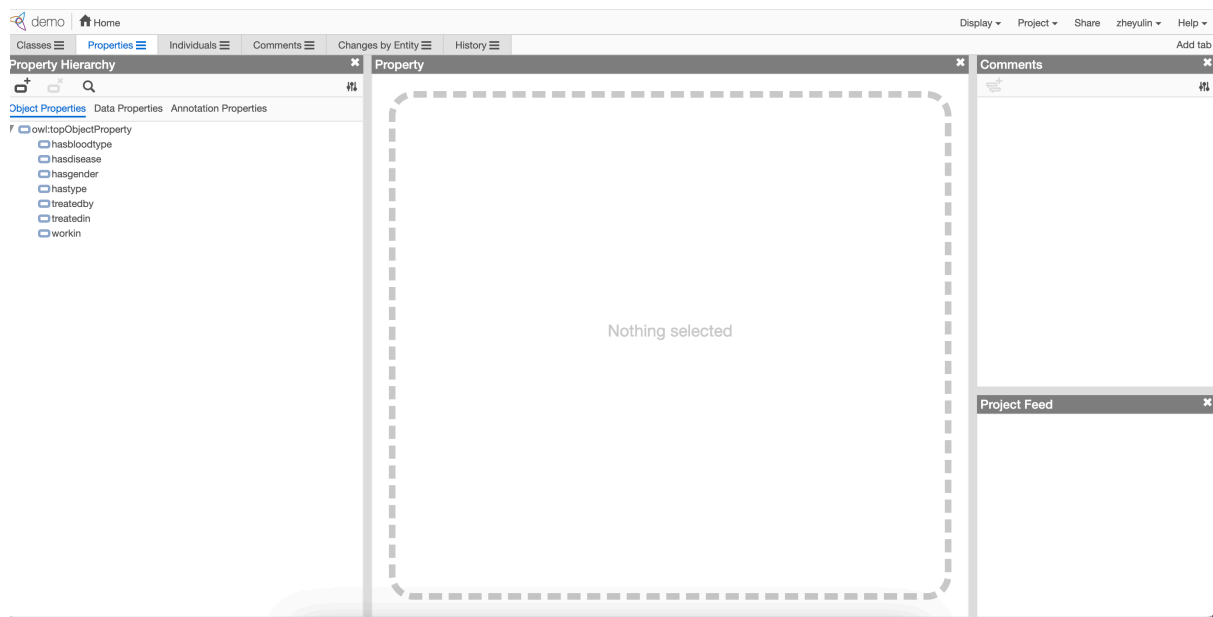


Protégé Desktop is a feature rich ontology editing environment with full support for the OWL 2 Web Ontology Language, and direct in-memory connections to description logic reasoners like *Hermit* and *Pellet*.

- ✓ W3C standards compliant
- ✓ Customizable user interface
- ✓ Visualization support

Based on the dataset defined in the database, I create my ontology.





After creation, I upload it to BioPortal.

thesisdemo
Last uploaded: January 4, 2025

Summary | Classes | Properties | Notes | Mappings | Widgets

Details

Acronym	THESISDEMO
Visibility	Public
Description	This ontology is for thesis demo presentation
Status	Alpha
Format	OWL
Language	English
Contact	zhengyulin, lzy8172022@163.com

Submissions +

Version	Released	Uploaded	Downloads
1.0 (Parsed, Indexed, Metrics, Annotator)	01/04/2025	01/04/2025	OWL CSV RDF/XML

Views of THESISDEMO +
No views of THESISDEMO available

Metrics ?

Classes	12
Individuals	0
Properties	9
Maximum depth	1
Maximum number of children	6
Average number of children	4
Classes with a single child	0
Classes with more than 25 children	0
Classes with no definition	11

Visits
We are still collecting data for THESISDEMO

Projects using THESISDEMO +
No projects using THESISDEMO

thesisdemo
Last uploaded: January 4, 2025

Summary | **Classes** | Properties | Notes | Mappings | Widgets

Jump to:

- Bloodtype
 - A
 - AB
 - B
 - O
- Disease
 - Cancer
 - Obesity
- Doctor
 - Gender
 - Hospital
 - Person

Details | Visualization | Notes (0) | Class Mappings (11)

Preferred Name	Bloodtype
Synonyms	
ID	http://webprotege.stanford.edu/R0Ev7dJKSfH04reuHxVRS
label	Bloodtype
prefLabel	Bloodtype
subClassOf	http://www.w3.org/2002/07/owl#Thing

PRODUCTS
[BioPortal](#)
[BioPortal REST API](#)
[BioPortal Virtual Appliance](#)
[NCBO Web Widgets](#)

SUPPORT
[Contact Us](#)
[Documentation](#)
[NCBO Wiki](#)

ABOUT
[About Us](#)
[Mission & Vision](#)
[Team](#)
[Projects](#)

CONNECT

Finally, download the ontology which will be used in next steps.

Mapping rules

Ontopic Studio can be found here

<https://ontopic.ai/en/ontopic-studio/>

Source database connection

I set up parameters connect Ontopic Studio and H2. This step ensure that Ontopic Studio could read the data stored in H2 automatically. Even if there are some data changes in H2, Ontopic Studio can read the updated data automatically.

The driver and username are same information used when log in the database.

Optionally, I add “FILE_LOCK=SOCKET ”at the end of the connection URL. It can ensure the file will not be locked during use.

demo2 > Settings

English

Dashboard

Ontology

Settings

Data source

Connection information

Connect by: Host ☐ URL ☒

Database: H2

Driver: org.h2.Driver

Username: Username

Password - optional: Insert Password - optional

Connection URL:

Test connection Save

Activate Windows
Go to Settings to activate Windows.

Data can be viewed in “Lenses” tab.

demo > Lenses > Lenses.TEST.PUBLIC.PATIENTINFO

English

Dashboard

Ontology

Lenses

Mapping

Search

Query

Settings

Create column Parent Conditions Constraints Restore/Remove columns

ID INTEGER	NAME CHARACTER V	AGE INTEGER	GENDER CHARACTER V	BLOODTYPE CHARACTER V	DISEASE CHARACTER V	DOCTOR CHARACTER V	HOSPITAL CHARACTER V
1	Bobby JacksOn	30	Male	B-	Cancer	Matthew Smith	Sons and Miller
2	Leslie TErRy	62	Male	A+	Cancer	Samantha Davies	Kim Inc
3	DaNnY sMiTh	76	Female	A-	Obesity	Tiffany Mitchell	Cook PLC

3 rows 8 columns

0 mapping entries

demo > Lenses >

lenses.TEST.PUBLIC.PATIENTINFO

English

Dashboard

Ontology

Lenses

Mapping

Search

Query

Settings

Create column

Parent

Conditions

Constraints

Restore/Remove columns

Column	#1	#2	#3
ID - INTEGER	1	2	3
NAME - CHARACTER VARYING	Bobby JacksOn	LesLie TErRy	DaNnY sMitH
AGE - INTEGER	30	62	76
GENDER - CHARACTER VARYING	Male	Male	Female
BLOODTYPE - CHARACTER VARYING	B-	A+	A-
DISEASE - CHARACTER VARYING	Cancer	Cancer	Obesity
DOCTOR - CHARACTER VARYING	Matthew Smith	Samantha Davies	Tiffany Mitchell
HOSPITAL - CHARACTER VARYING	Sons and Miller	Kim Inc	Cook PLC

30 rows 8 columns

0 mapping entries

Ontology upload

To set the mapping rules by mapping the source data to ontology, I upload the ontology created by myself into Ontopic Studio.

demo > Ontology > Class >

New class

English

Dashboard

Ontology

Lenses

Mapping

Search

Query

Settings

Classes

Properties

Prefix manager

Import ontology

Export ontology

Delete

Search

Import existing ontology

Please select an existing ontology file

Note: owl:imports is not supported (it is ignored)

Choose File

No file chosen

Select the ontology format

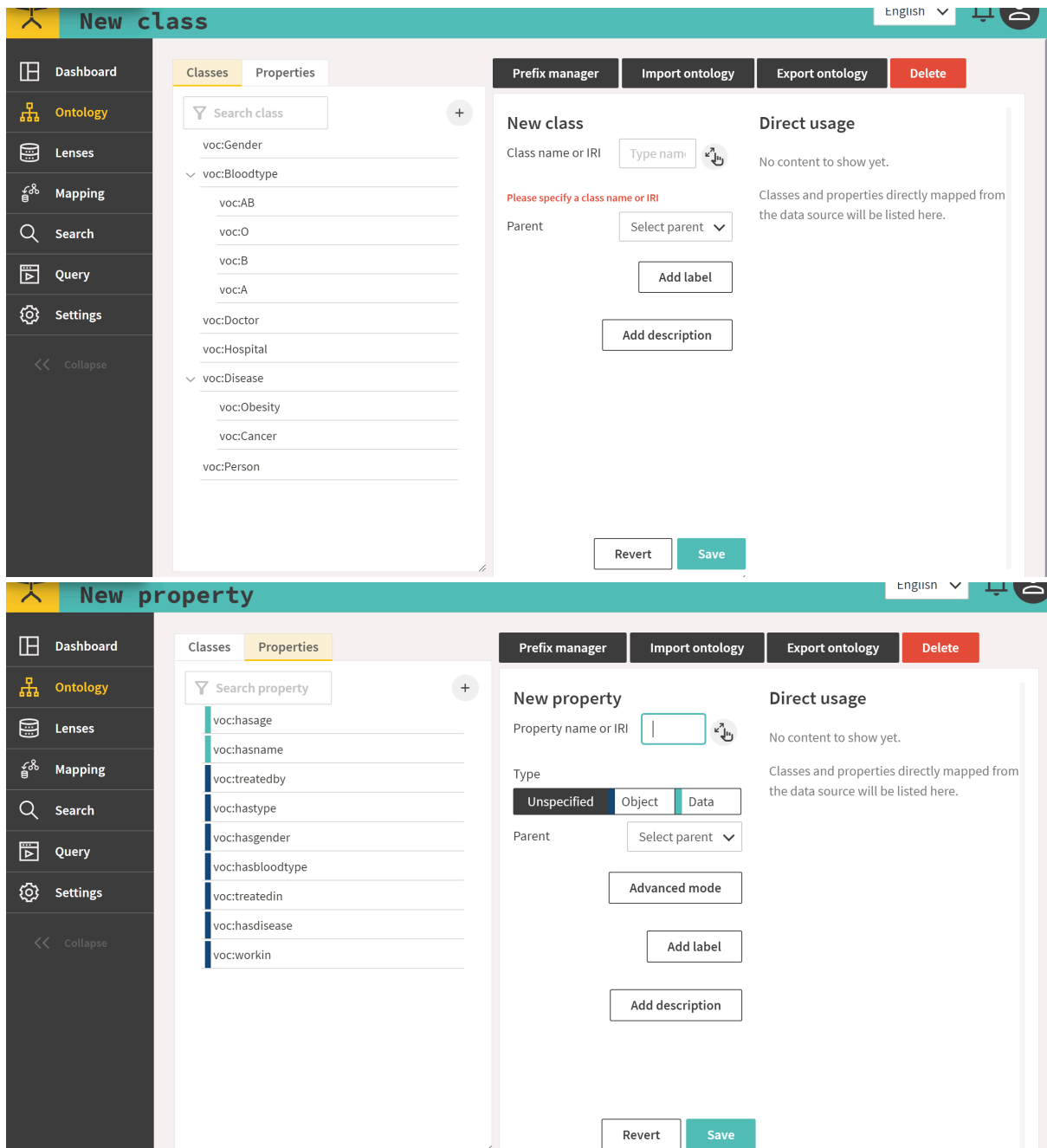
Select the ontology format

Cancel

Import

Revert

Save



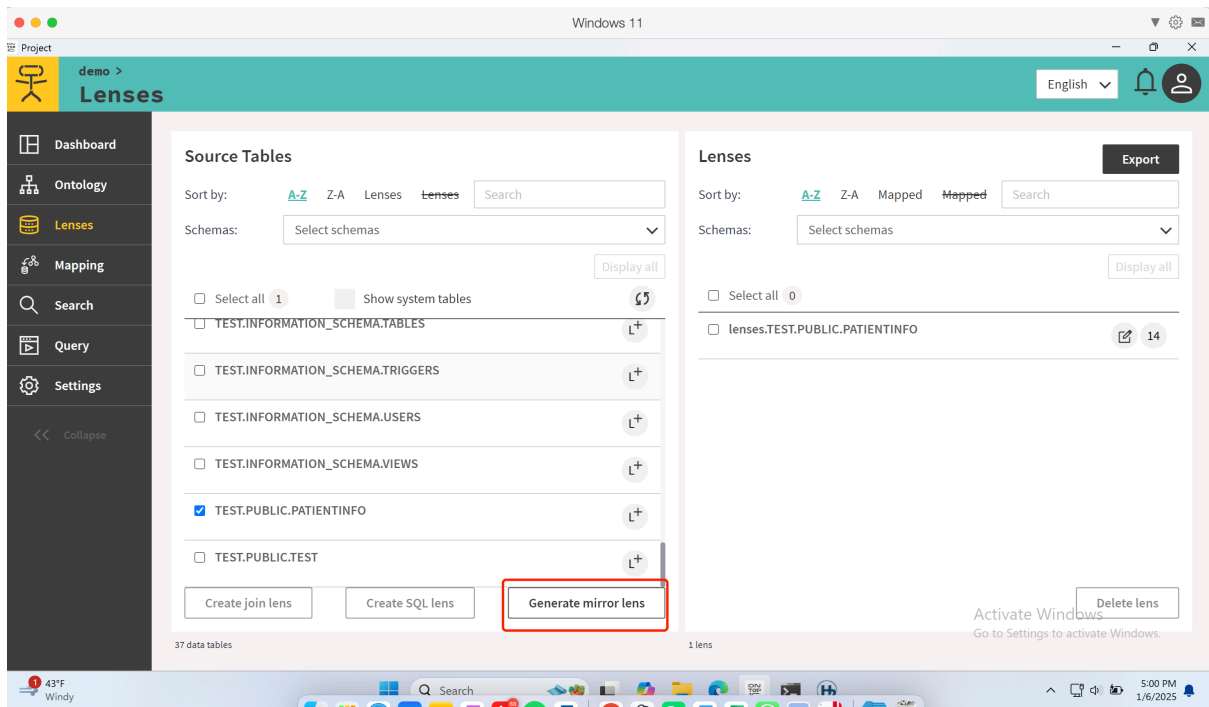
Set mapping rules

The official support document can be found here:

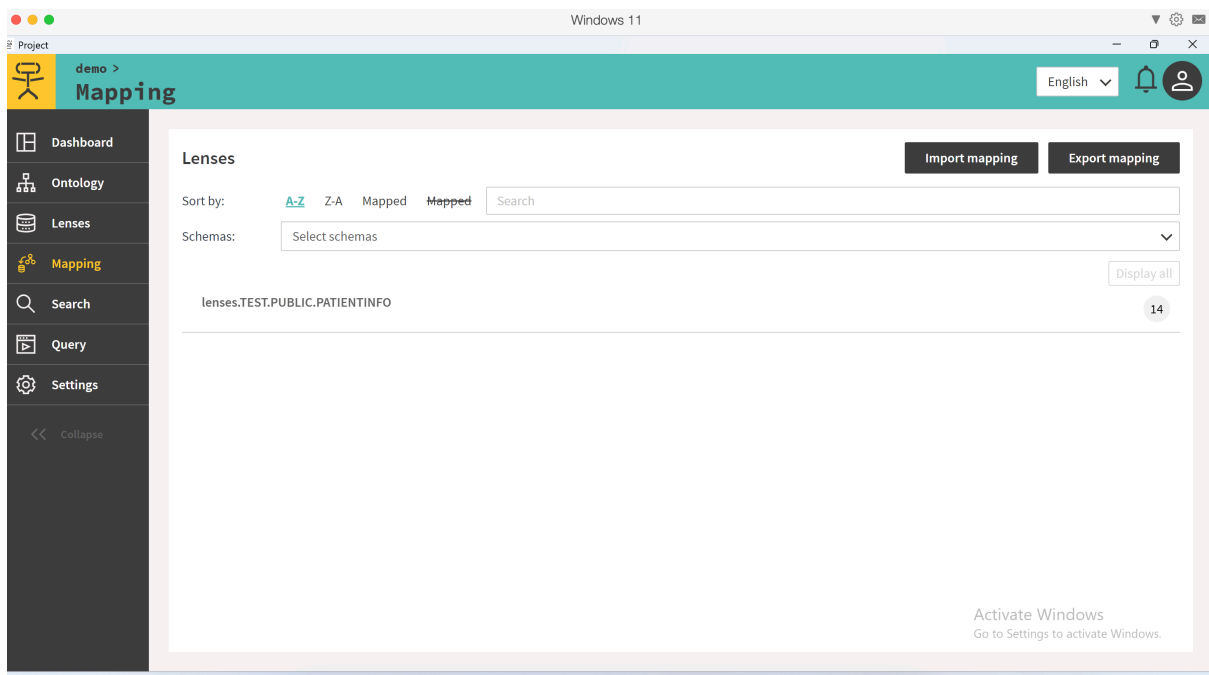
<https://docs.ontopic.ai/studio/learn/quickstart.html#creating-lenses>

Here is the example indicating how I set the mapping logic.

First, choose the dataset and click “Generate Mirror Lenses”.



Then, come to the “Mapping” tab and enter the dataset.



Then, click “C+” to create new template. The first class could also be defined at the same time.

Dashboard

Ontology

Lenses

Mapping

Search

Query

Settings

<< Collapse

Search

Comments

C⁺P⁺

No content to show yet.
Map classes and properties.

C⁺P⁺

Activate Windows

Go to Settings to activate Windows.

Dashboard

Ontology

Lenses

Mapping

Search

Query

Settings

<< Collapse

Subject

Type

TemplateConstantColumn

Template

Select or create a template

Please specify a template

Class

Type

ConstantDictionary

Class

Select class

Comment

Comment - optional

Add comment

Back

Save

Column ↑ ↓

CONSTRAINT_CATALOG - CHARACTER VARYING

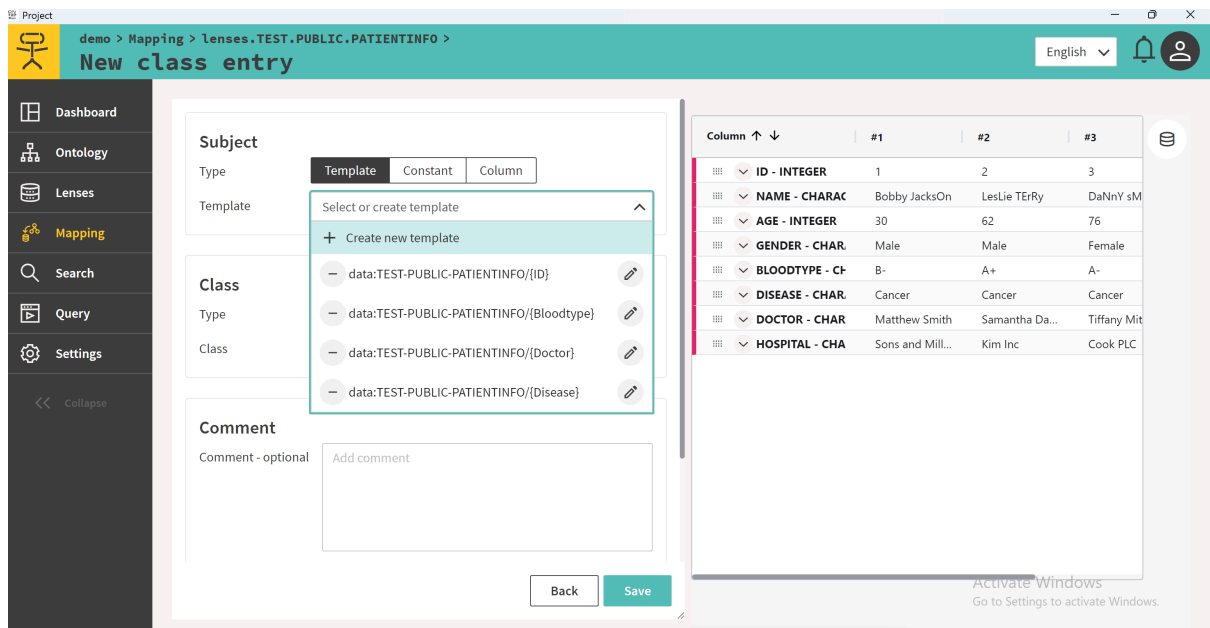
CONSTRAINT_SCHEMA - CHARACTER VARYING

CONSTRAINT_NAME - CHARACTER VARYING

CHECK_CLAUSE - CHARACTER VARYING

Activate Windows

Go to Settings to activate Windows.



Create template

data:TEST-PUBLIC-PATIENTINFO/{ID}



Text

Binding

data:TEST-PUBLIC-PATIENTINFO/

–

Text

Binding

ID

–

+

Cancel

Create

Then, I first create class by selecting the class “Person” in the ontology.

demo1 > Mapping > Lenses.TEST.PUBLIC.PATIENTS > **New class entry** English

Dashboard
Ontology
Lenses
Mapping
Search
Query
Settings

Subject
Type: Template Constant Column
Template: data:TEST-PUBLIC-PATIENTS/{ID}
ID: ID

Class
Type: Constant Dictionary
Class: Select class
foaf:OnlineEcommerceAccount
foaf:Organization
foaf:Person
foaf:Image
foaf:Agent
foaf:PersonalProfileDocument

Comment
Comment - optional

Column ↑ ↓	#1	#2	#3
ID - INTEGER	1	2	3
NAME - CHARAC	Bobby JacksOn	LesLie TErRy	DaNnY sM
AGE - INTEGER	30	62	76
GENDER - CHAR	Male	Male	Female
BLOOD_TYPE - C	B-	A+	A-
MEDICAL_COND	Cancer	Cancer	Obesit
DATE_OF_ADNIS	2024-01-31	2019-08-20	2022-C
DOCTOR - CHAR	Matthew Smith	Samantha Da...	Tiffany
HOSPITAL - CHA	Sons and Mill...	Kim Inc	Cook F

Back Save

Activate windows
Go to Settings to activate Windows.

Then, I create property by selecting the property “hasname” in the ontology.

demo > Mapping > Lenses.TEST.PUBLIC.PATIENTINFO > **Edit property entry** English

Dashboard
Ontology
Lenses
Mapping
Search
Query
Settings

Subject
Type: Template Constant Column
Template: data:TEST-PUBLIC-PATIENTINFO/{ID}
ID: ID

Property
Property: hasname
+ Create new property
hasage
hasname
treatedby
hastype
hasgender

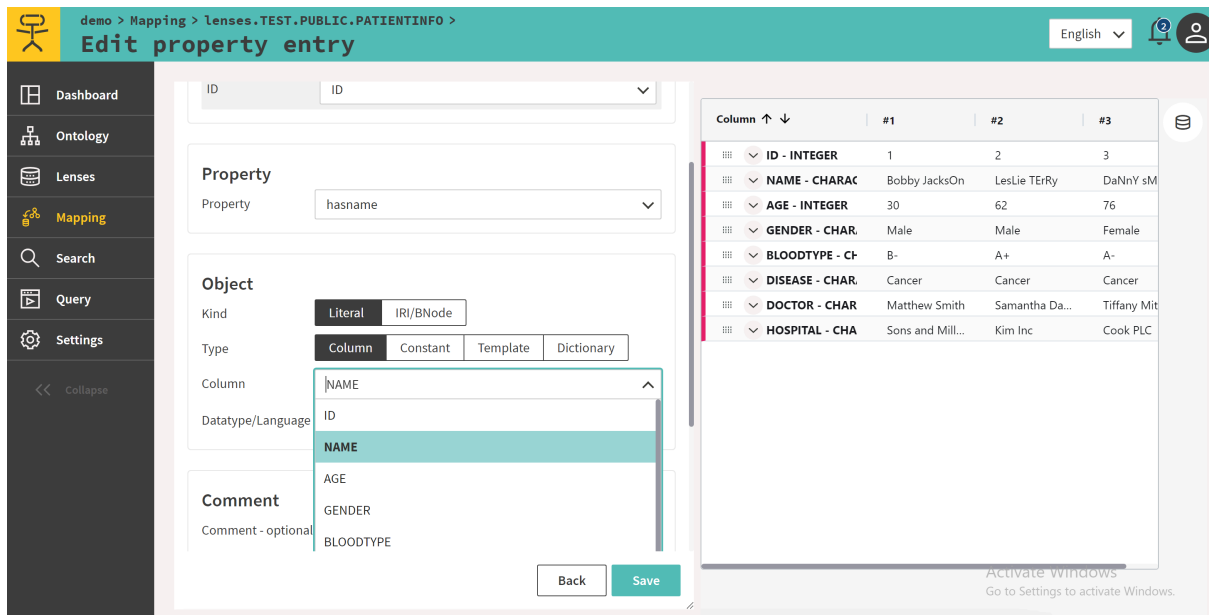
Object
Kind:
Type:
Column:

Column ↑ ↓	#1	#2	#3
ID - INTEGER	1	2	3
NAME - CHARAC	Bobby JacksOn	LesLie TErRy	DaNnY sM
AGE - INTEGER	30	62	76
GENDER - CHAR	Male	Male	Female
BLOODTYPE - CH	B-	A+	A-
DISEASE - CHAR	Cancer	Cancer	Cancer
DOCTOR - CHAR	Matthew Smith	Samantha Da...	Tiffany Mit
HOSPITAL - CHA	Sons and Mill...	Kim Inc	Cook PLC

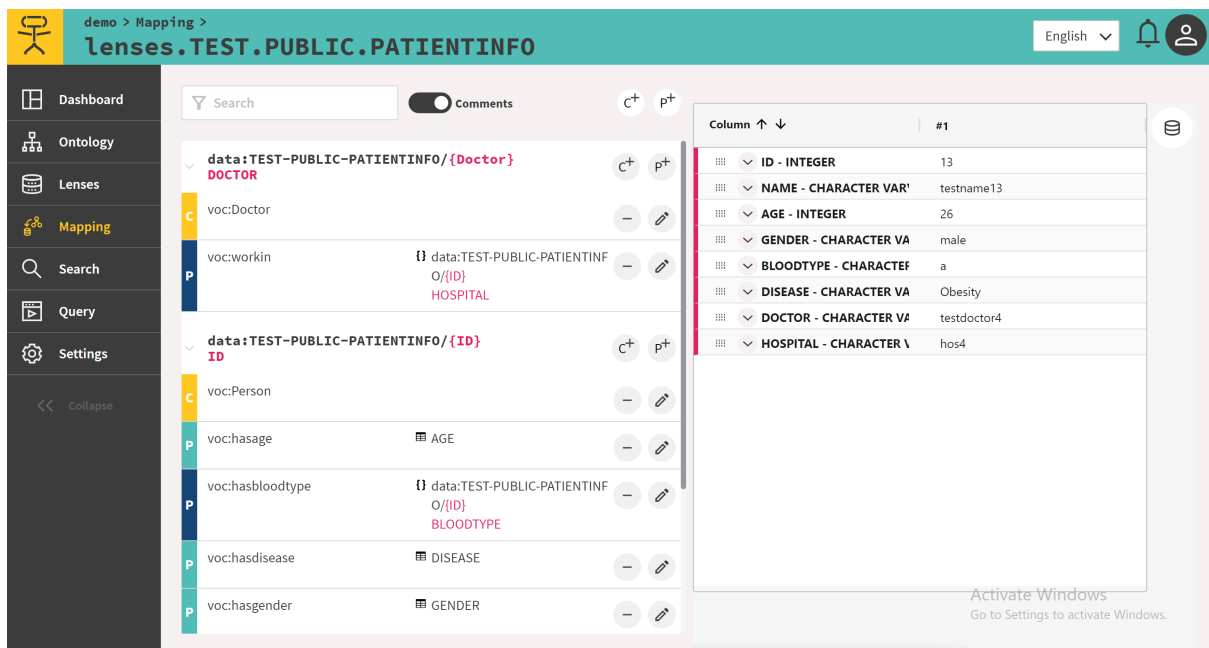
Back Save

Activate windows
Go to Settings to activate Windows.

Then, I select the column “NAME” in the source data as the object. This step lets the Ontopic Studio know where to find the value of name for each person during the RDF transformation.



Follow this logic, I map all source data to the ontology.



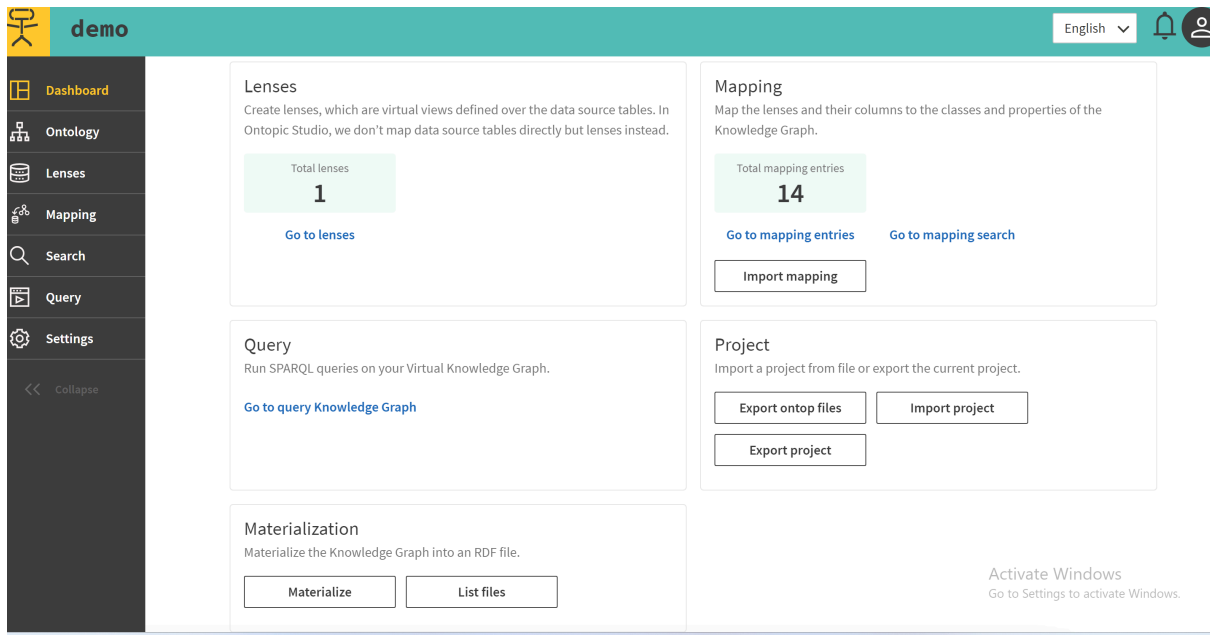
Materialize

Materialize the source data will transfer source data into RDF format.

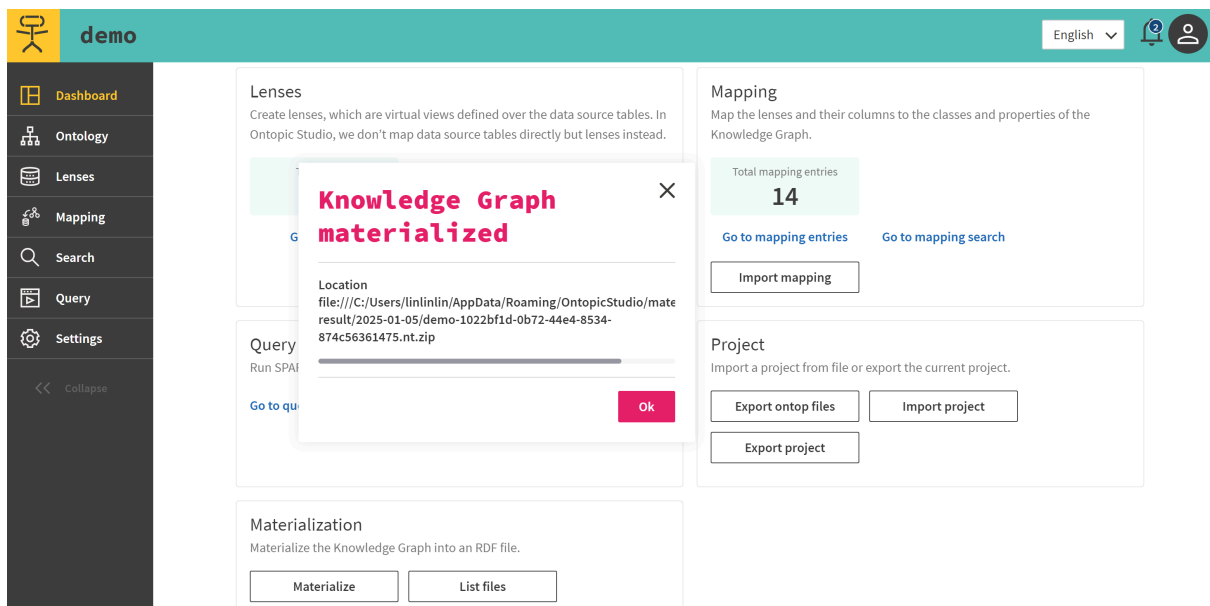
Manual way

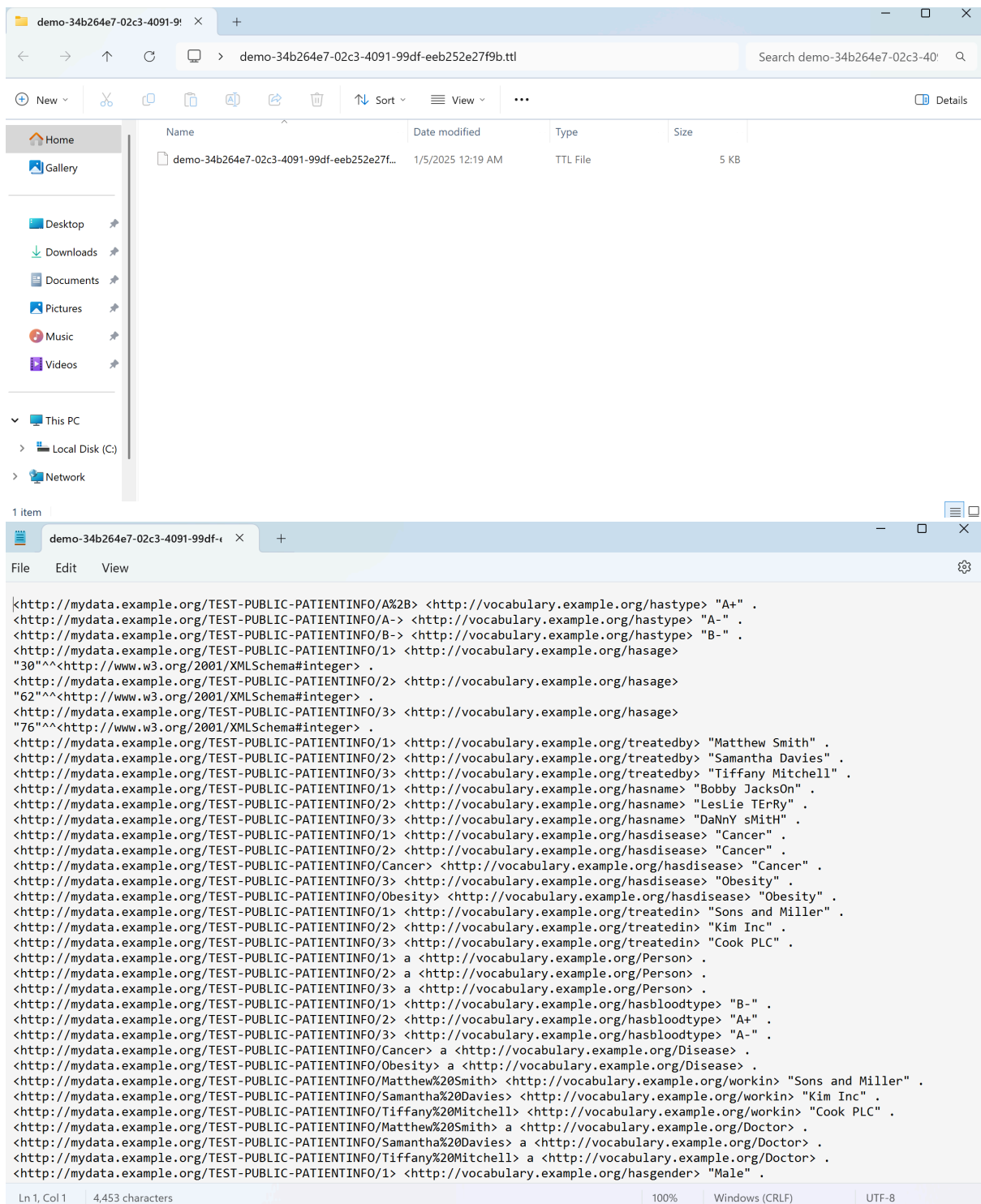
This method is simple and only requires clicking a button. However, this method requires manual operation.

Back to the “dashboard” tab after previous steps.



Click “Materialize”.





Automatic way

Users can use Ontop CLI-materialize to achieve the automation. Ontop itself does not have a complete graphical interface tool and mainly relies on the command line (CLI) and configuration files for operation.

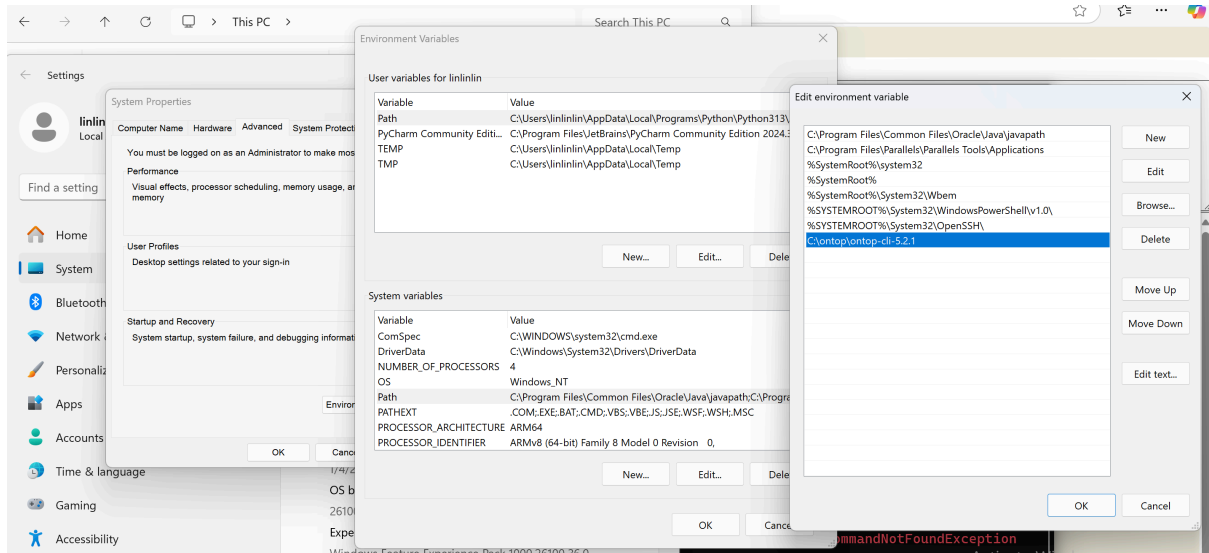
Ontop can be found here

<https://sourceforge.net/projects/ontop4obda/files/>

Set path

In this Example, my installation file path is "C:\ontop\ontop-cli-5.2.1"

So I set this as path.



Then, test whether it is correct use command “ontop” in Powershell (do not close the command prompt).

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\linlinlin> ontop
usage: ontop <command> [ <args> ]

Commands are:
--version          Show version of ontop
bootstrap          Bootstrap ontology and mapping from the database
endpoint           Start a SPARQL endpoint powered by Ontop
extract-db-metadata Extract the DB metadata and serialize it into an output JSON file
help              Display help information
materialize         Materialize the RDF graph exposed by the mapping and the OWL ontology
query             Query the RDF graph exposed by the mapping and the OWL ontology
validate          Validate Ontology and Mappings
mapping           Manipulate mapping files

See 'ontop help <command>' for more information on a specific command.
PS C:\Users\linlinlin> |
```

Create Ontop.properties

In the root directory of the ontop installation file, create a new file “ontop.properties”. Ensure the type is “properties file”.

In this file, write information:

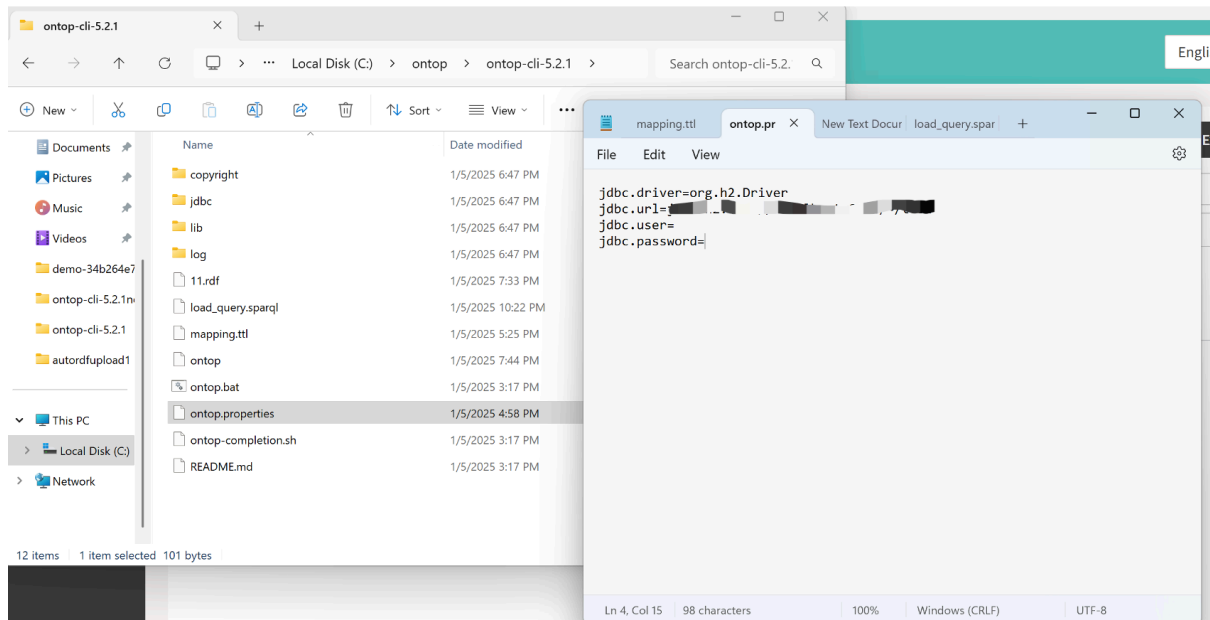
jdbc.driver=org.h2.Driver

```
jdbc.url='replace your own'
```

```
jdbc.user= 'replace your own'
```

```
jdbc.password='replace your own'
```

These information are the same with the information when log in H2 database.



Connect Ontop and database

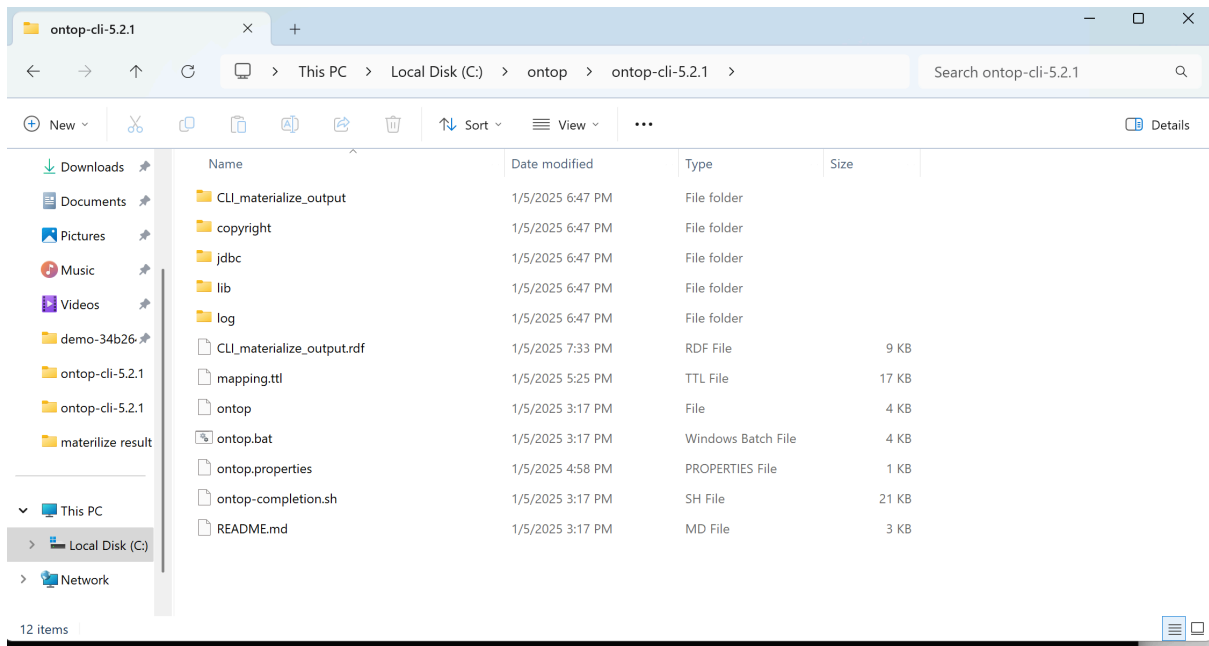
Still keep the command prompt, execute command in Powershell.

First enter the file, in this example,

```
cd C:\ontop\ontop-cli-5.2.1
```

Then, under this file, use command

```
ontop.bat extract-db-metadata -p "ontop.properties"
```

Materialize

Still keep the command prompt, execute command in Powershell.

First enter the file, in this example,

```
cd C:\ontop\ontop-cli-5.2.1
```

Then, under this file, use command

```
ontop materialize -m "mapping.ttl" -o "CLI_materialize_output" -p "ontop.properties"
```

- ontop materialize: This is a command in the Ontop command line interface (CLI) that performs a data materialization operation. The materialization process converts the data in the database into RDF format data according to the mapping rules.
- The -m parameter is followed by the path of the mapping file. In this example, it is "mapping.ttl"
- The -o parameter specifies the output file of the materialization operation. In this example, I use "CLI_materialize_output" as the file name of the output. Users can give another name. When you use the same name and execute this command many times, the output file will be automatically replaced with the new one. Multiple output files will only be retained if you set a different output file name each time you run the program.
- The -p parameter specifies a configuration file, which contains settings related to database connection, such as database URL, username, password, etc. In this example, "ontop.properties" is a configuration file containing database connection information which has been defined before.

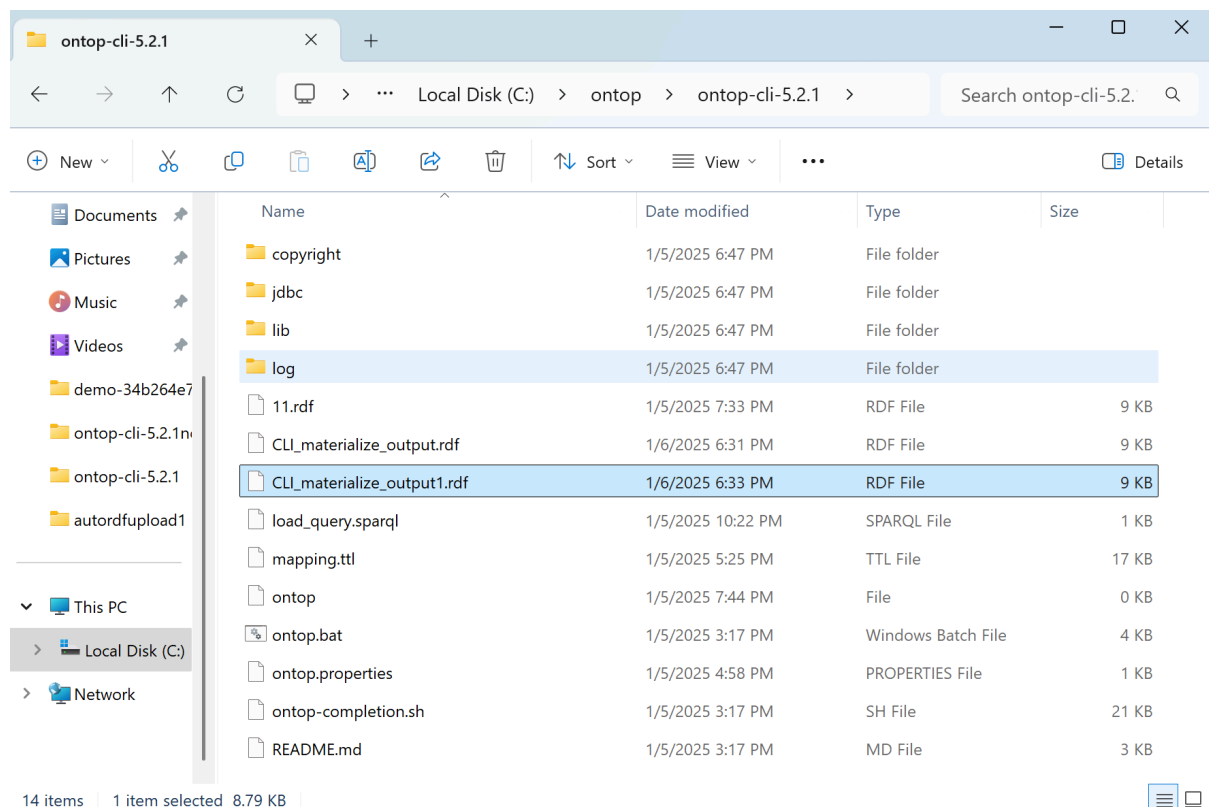
As shown in the picture, I ran it multiple times. The output files I set for the first three times had the same name "CLI_materialize_output", so there was only the latest version

in the folder. The fourth time, I set a different output file name

“CLI_materialize_output1”, so there were both historical output files and current output files in the folder.

```
PS C:\ontop\ontop-cli-5.2.1> ontop materialize -m "mapping.ttl" -o "CLI_materialize_output" -p "ontop.properties"
18:28:56.719 |-INFO in i.u.i.o.a.r.impl.QuestQueryProcessor - Ontop has completed the setup and it is ready for query a
nswering!
NR of TRIPLES: 50
Elapsed time to materialize: 884 {ms}
PS C:\ontop\ontop-cli-5.2.1> ontop materialize -m "mapping.ttl" -o "CLI_materialize_output" -p "ontop.properties"
18:29:29.454 |-INFO in i.u.i.o.a.r.impl.QuestQueryProcessor - Ontop has completed the setup and it is ready for query a
nswering!
NR of TRIPLES: 50
Elapsed time to materialize: 871 {ms}
PS C:\ontop\ontop-cli-5.2.1> ontop materialize -m "mapping.ttl" -o "CLI_materialize_output" -p "ontop.properties"
18:31:49.716 |-INFO in i.u.i.o.a.r.impl.QuestQueryProcessor - Ontop has completed the setup and it is ready for query a
nswering!
NR of TRIPLES: 50
Elapsed time to materialize: 877 {ms}
PS C:\ontop\ontop-cli-5.2.1> ontop materialize -m "mapping.ttl" -o "CLI_materialize_output1" -p "ontop.properties"
18:33:46.013 |-INFO in i.u.i.o.a.r.impl.QuestQueryProcessor - Ontop has completed the setup and it is ready for query a
nswering!
NR of TRIPLES: 50
Elapsed time to materialize: 808 {ms}
PS C:\ontop\ontop-cli-5.2.1>
```

Activate W
Go to Settings



Name	Date modified	Type	Size
copyright	1/5/2025 6:47 PM	File folder	
jdbc	1/5/2025 6:47 PM	File folder	
lib	1/5/2025 6:47 PM	File folder	
log	1/5/2025 6:47 PM	File folder	
11.rdf	1/5/2025 7:33 PM	RDF File	9 KB
CLI_materialize_output.rdf	1/6/2025 6:31 PM	RDF File	9 KB
CLI_materialize_output1.rdf	1/6/2025 6:33 PM	RDF File	9 KB
load_query.sparql	1/5/2025 10:22 PM	SPARQL File	1 KB
mapping.ttl	1/5/2025 5:25 PM	TTL File	17 KB
ontop	1/5/2025 7:44 PM	File	0 KB
ontop.bat	1/5/2025 3:17 PM	Windows Batch File	4 KB
ontop.properties	1/5/2025 4:58 PM	PROPERTIES File	1 KB
ontop-completion.sh	1/5/2025 3:17 PM	SH File	21 KB
README.md	1/5/2025 3:17 PM	MD File	3 KB

Upload RDF into triplestore

In this example, I use AllegroGraph, which can be found here:

<https://allegrograph.com/>

Manual way

Users can upload the RDF file into AllegroGraph manually.

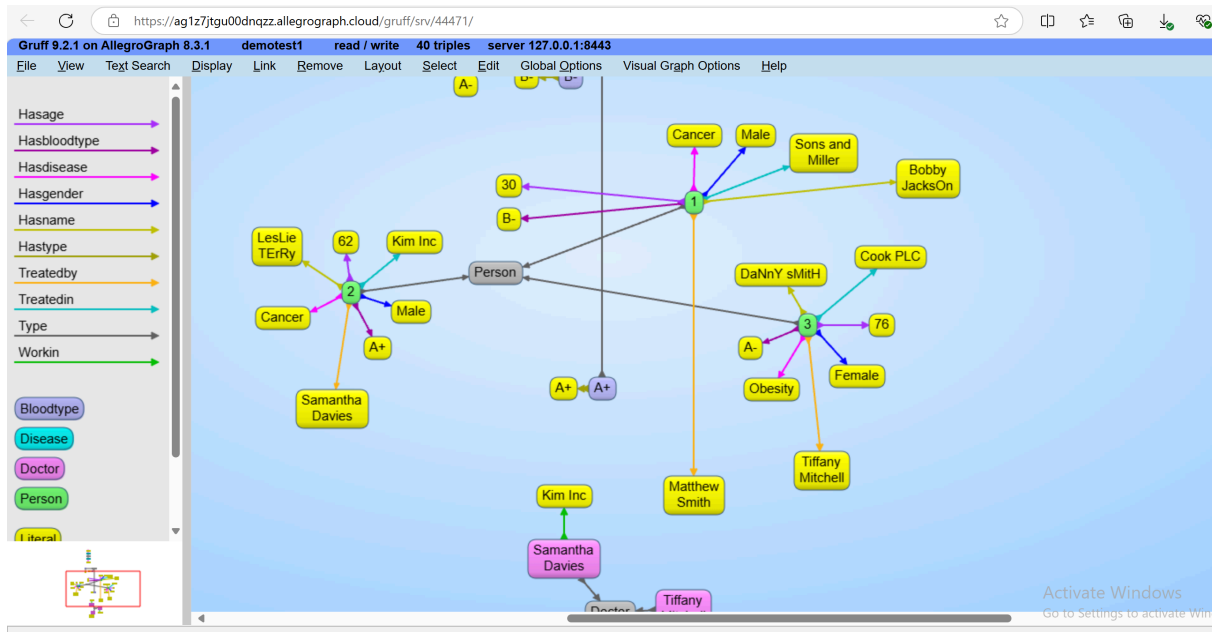
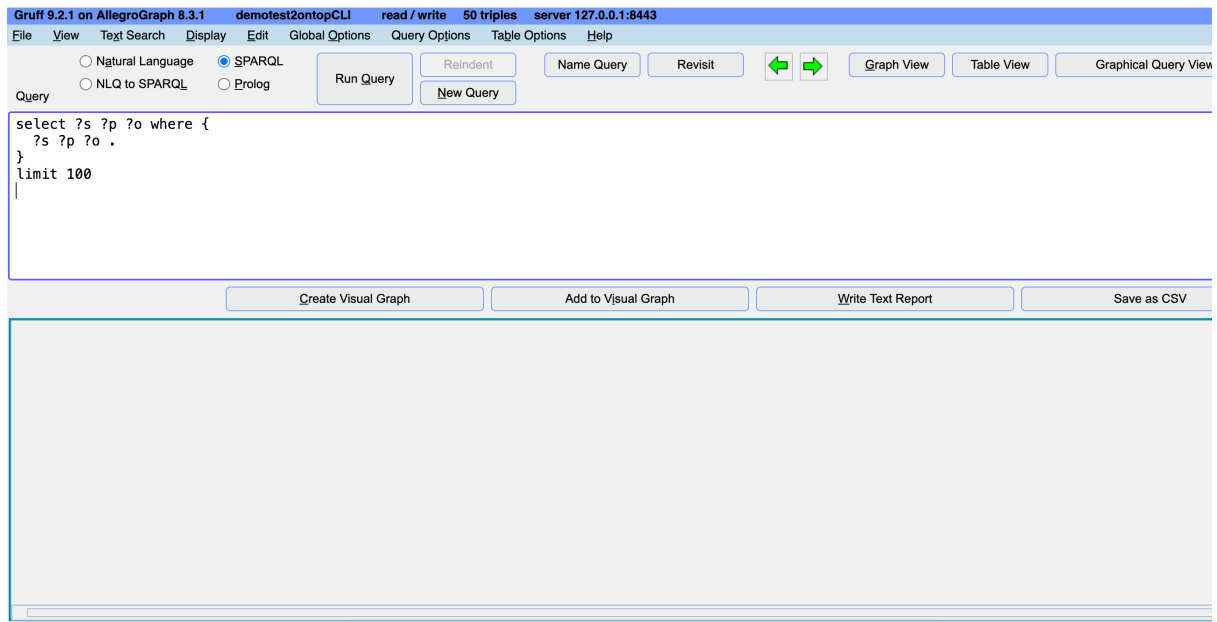
This RDF file could be the one downloaded from Ontopic Studio or the one generated by Ontop CLI-materialize.

The screenshot shows the AllegroGraph 8.3.1 web interface. The top navigation bar includes 'Catalog' (root), 'Repository' (test), and 'Statements' (0). The right side shows 'Server demo', a help icon, a user icon 'A', and 'ADMIN'. The left sidebar contains a tree view with 'root' selected. The main area is titled 'Select file to import' and features a dashed box for 'Drag-N-Drop an RDF file or click here to select files.' with a note: 'Note: gz, bz2 archives extensions are supported'. Below this is a 'VERIFY & IMPORT FILE' button and an 'Import Options' dropdown. To the right, the 'Input specification' section explains RDF and lists supported formats: N-Triples (.nt), N-Quads (.nq), Extended N-Quads (.nqx), RDF/XML (.xml), TriG (.trig), Turtle (.ttl), and JSON-LD (.json). Examples provided include [kennedy.nt](#) (1.2K triples), [actors.ntriples.gz](#) (116.5K triples), and [healthcare.nt.gz](#) (13.1M triples, paid license).

Then, in the Gruff, change the view to “Query view”.

The screenshot shows the Gruff 9.2.1 application window running on AllegroGraph 8.3.1. The title bar indicates 'test', 'read / write', '0 triples', and 'server 127.0.0.1:8443'. The 'View' menu is open, showing options: Graph View (G), Optional Graph View Panes, Table View (B), Outline View (O), Query View (W) - which is highlighted, Graphical Query View (E), Go Back (Z), Go Forward (Y), Copy (⌘C), Paste (⌘V), Add Selected Node to Outline (⇧O), Web Interaction, View Current Store in AGWebView (⇧⌘W), and Browse a Single Graph ... (⇧⌘G). A text box on the right provides instructions: 'To begin in the graph view, use a command on the Display menu (or the Text Search menu) to display a starter node. Then display nodes that are linked with the selected node by using commands on the Link menu. If you don't know much about the triple-store, then the command "Display Some Sample Triples" near the bottom of the Display menu will show some nodes to start with. Double-click a node to display its properties in the table view. At any time, right-click an object or the background (or press M) for a pop-up menu of commands. For help on any menu command, highlight the command and press F1.' A red rectangle is visible in the bottom left corner of the main area.

First click “Run Query” and then “Creat Visual Graph”.



If users would like to query information, users could go back to the query view.

SELECT ?s ?p ?o

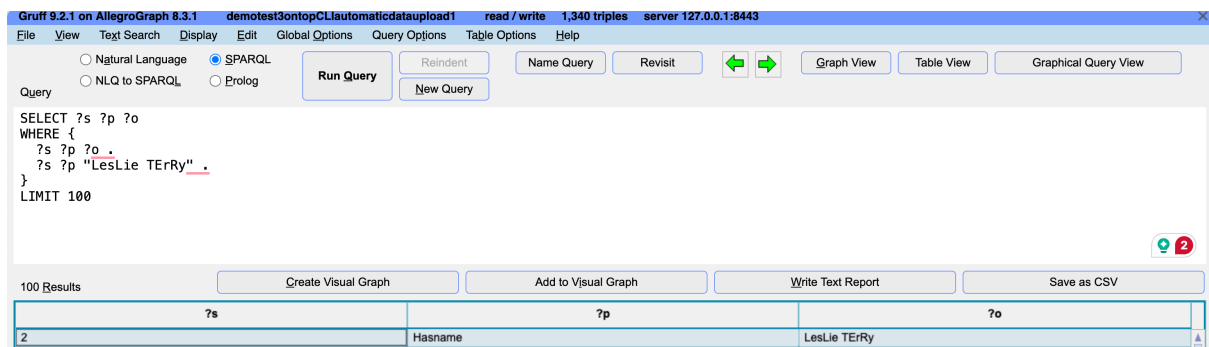
WHERE {

?s ?p ?o .

?s ?p "LesLie TErRy" .

}

LIMIT 100



Automatic data upload

Upload data to AllegroGraph through the REST API via HTTP POST requests, thereby achieving automatic data upload.

I write a python script that does this. In this way, I automatically upload the RDF file generated by Ontop CLI-materialize. This code just implements automatic uploading of RDF to AllegroGraph.

De novo FAIRification AutoFlow

To fully automate the process without human involvement, I wrote a Python script that integrates Ontop CLI-materialize with code to automatically upload RDF data to the database. This enables users to simply execute the script, which handles the entire workflow of materializing RDF data and uploading it to the database automatically, eliminating the need for manual intervention. And this code implements real-time RDF conversion of incremental data and uploads it to AllegroGraph. This design is more in line with the needs of the legacy system.

System Settings

Adjust PowerShell's execution policy

Allow PowerShell to execute scripts: You need to set PowerShell to allow execution of local scripts (including .ps1 scripts) and .bat files. This command allows you to execute local .ps1 scripts (for example, activate.ps1 files to activate virtual environments).

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

- RemoteSigned: Allows running local scripts (without signatures), but scripts downloaded from the Internet must be signed. This setting is relatively safe
- CurrentUser: Only affects the PowerShell execution policy of the current user.

Confirm that the changes have taken effect: You can confirm the current execution policy with the following command:

Get-ExecutionPolicy -List

If you see that the policy of CurrentUser is RemoteSigned, it means that it has been successfully modified.

Activate the virtual environment

After modifying the execution policy, you can try to activate the virtual environment again:

```
.\venv\Scripts\activate
```

```
PS C:\Users\linlinlin\PycharmProjects\autordfupload1> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned
-Scope CurrentUser
PS C:\Users\linlinlin\PycharmProjects\autordfupload1> Get-ExecutionPolicy -List

Scope ExecutionPolicy
-----
MachinePolicy Undefined
UserPolicy Undefined
Process Undefined
CurrentUser RemoteSigned
LocalMachine Undefined

PS C:\Users\linlinlin\PycharmProjects\autordfupload1> .\venv\Scripts\activate
(.venv) PS C:\Users\linlinlin\PycharmProjects\autordfupload1> |
```

Activate Windows
Go to Settings to activate Windows.

Then activate the virtual environment in PyCharm's terminal

```
.\venv\Scripts\activate
```

JPyype

Install, reinstall or upgrade JPyype in PyCharm's terminal

```
pip install --upgrade --force-reinstall jpype1
```

Check the JPyype path (sometimes JPyype is not installed in the correct location, which results in the inability to find the relevant DLL files. You can confirm this by looking at the JPyype installation path)

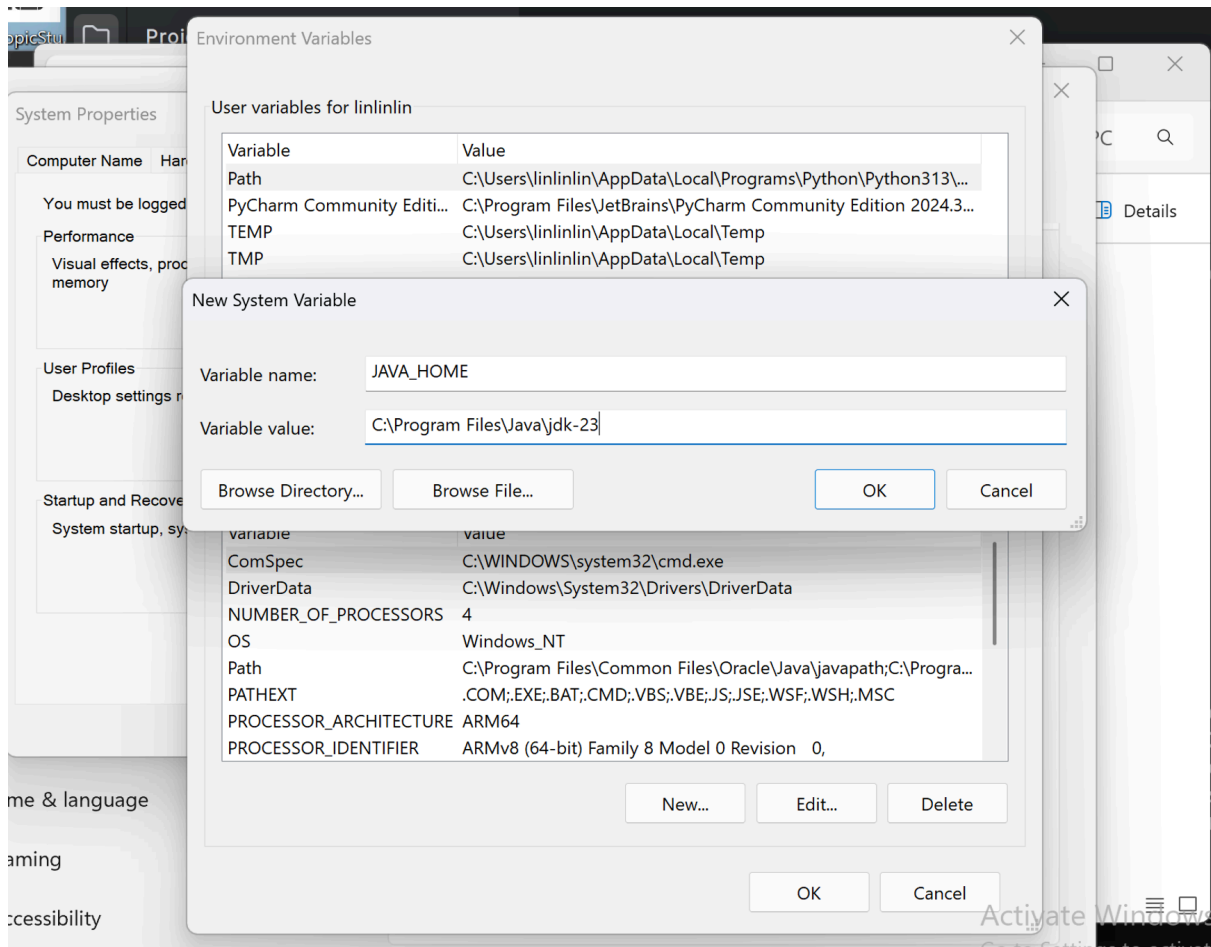
```
pip show jpype1
```

Java Configuration

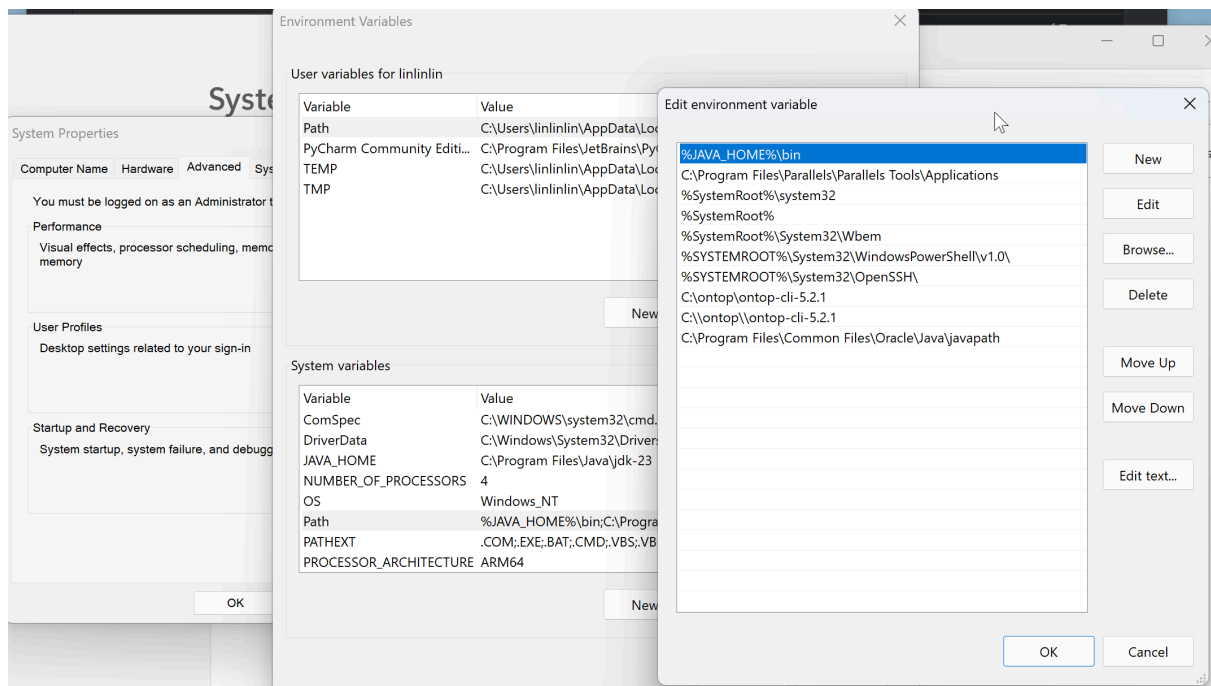
Check if Java is installed

```
java -version
```

Set the Java home system variable



Set the path, and set it first.



Specify the JDK path

Force the use of the correct JDK path: Manually specify the JDK path in the command line

```
"C:\Program Files\Java\jdk-23\bin\java" -version
```

```
C:\Users\linlinlin>"C:\Program Files\Java\jdk-23\bin\java" -version
java version "23.0.1" 2024-10-15
Java(TM) SE Runtime Environment (build 23.0.1+11-39)
Java HotSpot(TM) 64-Bit Server VM (build 23.0.1+11-39, mixed mode, sharing)

C:\Users\linlinlin>
```

Install Microsoft Visual C++ Redistributable

Install Microsoft Visual C++ Redistributable which can be found here:

<https://learn.microsoft.com/en-us/cpp/windows/latest-supported-vc-redist?view=msvc-170>

Source database modification

Add IS_PROCESSED in the source database. The default value of IS_PROCESSED for the newly added data is FALSE.

In this example,

```
ALTER TABLE allpatientinfo ADD IS_PROCESSED BOOLEAN DEFAULT FALSE;
```

IS_PROCESSED of the data that has been converted to RDF will become TRUE (this operation will be implemented in the Python script).

No incremental data

If the data in the database has been converted to RDF before, then the IS_PROCESSED of the data in the database will become TRUE. The IS_PROCESSED of the newly added data defaults to FALSE. The script will automatically check whether there is incremental data. If it is found that there is no incremental data, the materialize operation will not be performed. The return is "No new unprocessed data, skip materialize operation".

jdbc:h2:tcp://localhost:8083/~test

Max rows: 1000

Auto commit: ☒ Auto complete: ☐ Auto select: ☐

Run Selected Auto complete Clear SQL statement:

select * from allpatientinfo

select * from allpatientinfo;

ID	NAME	AGE	GENDER	BLOODTYPE	DISEASE	DOCTOR	HOSPITAL	UPDATED_AT	IS_PROCESSED
1	testname1	66	male	a	Obesity	testdoctor1	testhos1	2025-01-09 19:42:36.611799	TRUE
2	testname2	66	male	a	Obesity	testdoctor2	testhos2	2025-01-09 19:43:07.185092	TRUE
3	testname3	66	female	B	Cancer	testdoctor1	testhos1	2025-01-09 19:47:33.059304	TRUE
4	testname4	23	female	B	hypertension	testdoctor1	testhos1	2025-01-09 19:51:56.485036	TRUE
5	testname5	23	female	O	hypertension	testdoctor2	testhos2	2025-01-09 19:52:12.190022	TRUE
6	testname6	23	female	AB	diabetes	testdoctor3	testhos3	2025-01-09 19:53:01.510513	TRUE
7	testname7	47	male	AB	diabetes	testdoctor4	testhos2	2025-01-09 19:53:27.826403	TRUE
8	testname8	66	male	a	Obesity	testdoctor	testhos	2025-01-12 16:33:34.003433	TRUE
9	testname9	66	male	a	Obesity	testdoctor	testhos	2025-01-12 16:40:53.840015	TRUE
10	testname10	66	male	a	Obesity	testdoctor	testhos	2025-01-12 16:44:45.563181	TRUE
11	testname11	16	male	B	Obesity	testdoctor3	testhos	2025-01-12 16:49:31.663409	TRUE
12	testname12	66	male	a	Obesity	testdoctor2	testhos	2025-01-12 16:51:54.05424	TRUE

(12 rows, 213 ms)

Edit

Activate Windows
Go to Settings to activate Windows.

Project: autordfupload1

de-novo-v2.py

```

34 def copy_unprocessed_data():
35     conn = jaydebeapi.connect(jdbc_driver, jdbc_url, driver_args=[jdbc_user, jdbc_password], _t
36     cursor = conn.cursor()
37
38     # 查询 allpatientinfo 表中的 IS_PROCESSED 为 FALSE 的记录
39     cursor.execute("SELECT * FROM ALLPATIENTINFO WHERE IS_PROCESSED = FALSE")
40     unprocessed_data = cursor.fetchall() # 获取未处理的数据
41
42     # 如果没有未处理数据, 则直接返回
43     if not unprocessed_data:
44         print("No unprocessed data found in ALLPATIENTINFO, skip materialize operation")
45         return False
46
47     # 将未处理的数据复制到 patientinfo 表
48     for row in unprocessed_data:
49         cursor.execute("""
50             INSERT INTO PATIENTINFO (ID, NAME, AGE, GENDER, BLOODTYPE, DISEASE, DOCTOR, HOSP
51             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
52
53
54

```

Run: de-novo-v2

C:\Users\linlinlin\PycharmProjects\autordfupload1\venv\Scripts\python.exe C:\Users\linlinlin\PycharmProjects\autordfupload1\de-novo-
No unprocessed data found in ALLPATIENTINFO, skip materialize operation

Activate Windows
Go to Settings to activate Windows.

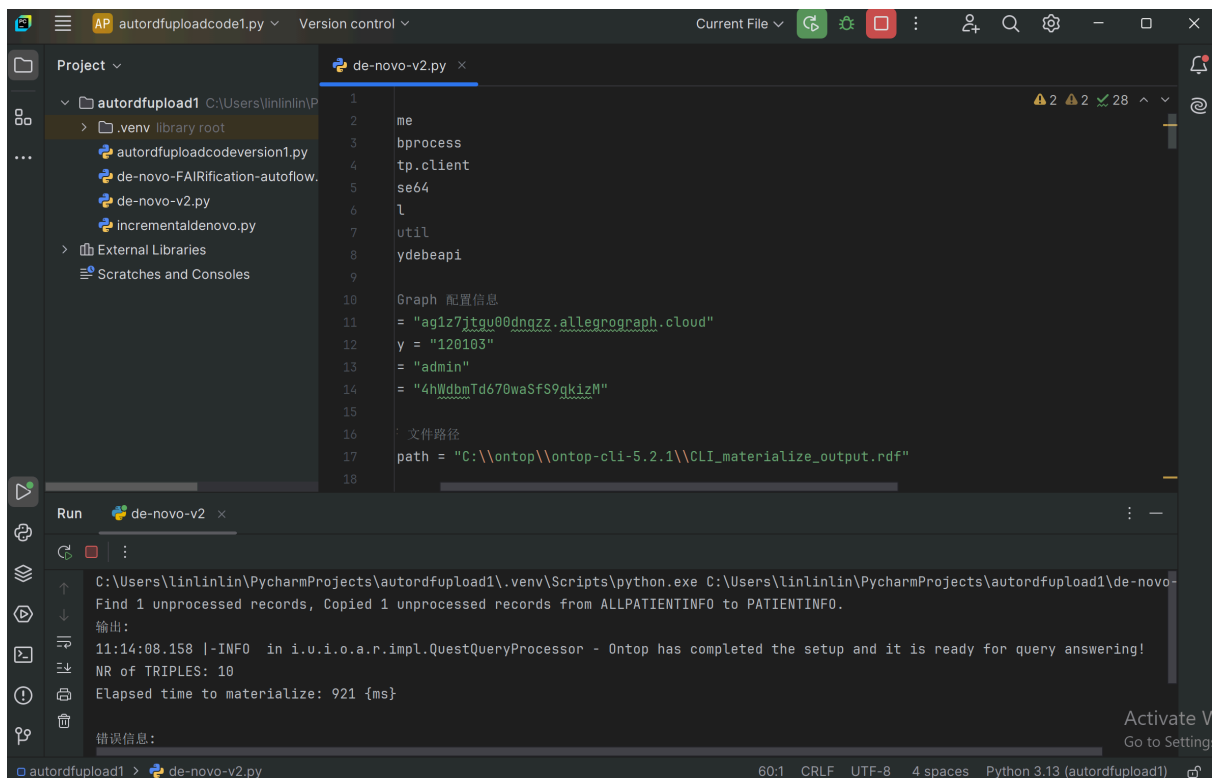
Incremental data

In this example, insert new data

INSERT INTO allpatientinfo (ID, NAME, AGE, GENDER, BLOODTYPE, DISEASE, DOCTOR, HOSPITAL)

VALUES (7, 'testnameC', 66, 'male', 'a', 'Obesity', 'testdoctor', 'testhos');

When the script is running and detects new unprocessed data, it will perform the materialize operation and return information such as how many incremental data were found.



The screenshot displays the PyCharm IDE interface. The main editor window shows the file `de-novo-v2.py` with the following Python code:

```
1 me
2 bprocess
3 tp.client
4 se64
5 l
6 util
7 ydebeapi
8
9
10 Graph 配置信息
11 = "ag1z7jtg00dnqzz.allegrograph.cloud"
12 y = "120103"
13 = "admin"
14 = "4hWdbmTd670waSfS9qkizM"
15
16 文件路径
17 path = "C:\\ontop\\ontop-cli-5.2.1\\CLI_materialize_output.rdf"
18
```

The Run console at the bottom shows the execution output for `de-novo-v2`:

```
C:\Users\linlinlin\PycharmProjects\autordfupload1\.venv\Scripts\python.exe C:\Users\linlinlin\PycharmProjects\autordfupload1\de-novo-
Find 1 unprocessed records, Copied 1 unprocessed records from ALLPATIENTINFO to PATIENTINFO.
输出:
11:14:08.158 |-INFO in i.u.i.o.a.r.impl.QuestQueryProcessor - Ontop has completed the setup and it is ready for query answering!
NR of TRIPLES: 10
Elapsed time to materialize: 921 {ms}
错误信息:
```

The status bar at the bottom indicates the file encoding is UTF-8, 4 spaces, and the Python interpreter is Python 3.13 (autordfupload1).

Real-time monitoring

The script I wrote is currently executed once a minute to ensure that data can be converted to RDF in real time, thus achieving de novo FAIRification. This parameter can also be adjusted.